1. TYPES OF KEYS IN RELATIONAL MODEL

1. INTRODUCTION

- Keys are essential elements in DBMS.
- They help identify records uniquely, establish relationships between tables, ensure data consistency & integrity and enable faster data retrieval.
- Keys are defined at table level and are based on functional dependency & uniqueness.

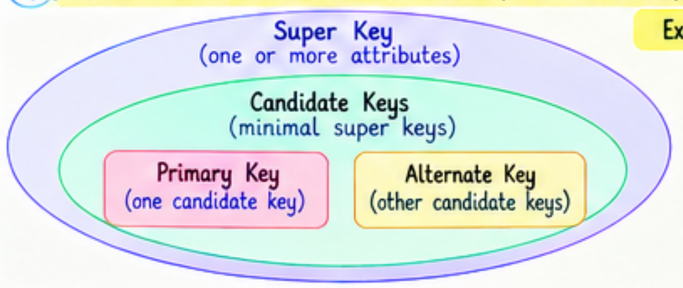
2. IMPORTANCE / CHARACTERISTICS

- ✓ **Uniqueness** – helps identify each record uniquely.
- ✓ **Data integrity** – prevents duplicate/invalid data.
- ✓ **Relationships** – helps establish links between tables.
- ✓ **Faster data retrieval** – can be indexed for quick lookup.
- ✓ Helps maintain referential integrity.

3. TYPES OF KEYS (OVERVIEW)

Key Type	Definition	Key Points	Example
Super Key	A set of one or more attributes that uniquely identifies a record.	• May contain extra attributes. • Can have multiple super keys.	{employee_id} {employee_id, name}
Candidate Key	Minimal super key (no unnecessary attributes).	• Uniquely identifies a record. • Multiple candidate keys can exist.	{employee_id} {email}
Primary Key	One candidate key chosen to uniquely identify records.	• Must be unique & NOT NULL. • Only one primary key per table.	employee_id
Alternate Key	A candidate key which is not chosen as primary key.	• Uniquely identifies a record. • Enforced using UNIQUE constraint.	email (if not PK)
Unique Key	Constraint to ensure uniqueness of column(s).	• No duplicate values allowed. • NULL handling depends on DBMS (1 or multiple NULLs may be allowed).	UNIQUE(email)
Composite Key	Combination of two or more columns to uniquely identify a record.	• Each column alone is not unique, but together they are.	{student_id, course_id}
Foreign Key	Attribute(s) that reference a primary/unique key in another table.	• Ensures referential integrity. • Can reference a primary key or candidate/unique key.	orders.customer_id → customers.customer_id
Partial Key	Identifies a weak entity within its parent entity.	• Unique only within the parent. • Used with composite key.	dependent_no (with employee_id)
Secondary Key	Attribute used for searching and indexing, not necessarily unique.	• Helps in faster retrieval. • Can return multiple records.	city, department
Surrogate Key	Artificial key generated by system/application, with no business meaning.	• Usually integer/UUID. • Independent of business data. • Used when natural key is not suitable.	customer_sk = 501
Natural Key	Business meaningful key from source system.	• Has real-world meaning. • May change over time (not always stable).	customer_id = C101

4. RELATIONSHIP BETWEEN SUPER, CANDIDATE, PRIMARY & ALTERNATE KEYS



Example

Employee Table:

employee_id	email	department
101	a@company.com	IT
102	b@company.com	Finance
103	c@company.com	IT

- Super Keys: {employee_id}, {email}, {employee_id, department}
- Candidate Keys: {employee_id}, {email}
- Primary Key: employee_id
- Alternate Key: email

2. SURROGATE KEY & PRACTICAL CONCEPTS

1. SURROGATE KEY OVERVIEW

- A surrogate key is an artificial key generated by the system or application.
- It has no business meaning and is usually an integer value (can also be UUID).
- Used when natural key is not suitable (e.g. multiple source systems, changing business attributes, or need to maintain historical versions).

2. ADVANTAGES & DISADVANTAGES

Advantages

- ✓ Stable and does not change.
- ✓ Independent of business data.
- ✓ Supports SCD Type 2 history.
- ✓ Simplifies relationships.
- ✓ Can use compact integers.
- ✓ Helps integrate multiple sources.
- ✓ Good for indexing and joins.

Disadvantages

- ✗ No business meaning.
- ✗ Additional storage space (extra column).
- ✗ Requires key generation and mapping logic.
- ✗ Cannot infer business identity from surrogate alone.
- ✗ Performance benefit depends on proper indexing.

3. SURROGATE KEY IN SCD TYPE 2 (EXAMPLE)

customer_sk	customer_id	name	city	effective_from	effective_to	is_current
501	C101	Rahul	Delhi	2024-01-01	2025-01-01	N
502	C101	Rahul	Bengaluru	2025-01-01	9999-12-31	Y

Key Takeaways

- customer_id is the business (natural) key. customer_sk is the surrogate key.
- Surrogate key allows multiple historical versions for the same customer_id.
- Each version has a unique surrogate key.

4. SURROGATE KEY GENERATION METHODS

- Identity/Auto Increment (e.g. IDENTITY, SEQUENCE)
- UUID/GUID (e.g. uuid())
- Hash-based key (e.g. hash of business key)
- Generated in ETL/ELT pipeline using window functions or MERGE logic

Example (SQL - Using Sequence)

```
INSERT INTO dim_customer (customer_sk, customer_id, name)
    SELECT NEXTVAL('customer_sk_seq') AS customer_sk,
        customer_id, name
    FROM staging_customer;
```

5. NATURAL KEY vs SURROGATE KEY (COMPARISON)

Aspect	Natural Key	Surrogate Key
Meaning	Business identifier	No business meaning
Source	From source system	Generated by system
Stability	May change	Stable (does not change)
Example	customer_id, email	customer_sk (501)
Use Case	OLTP, business lookups	Data warehouse, SCD, integration
Pros	Meaningful, no extra column	Stable, supports history, simpler
Cons	May change, complex joins	Extra storage, no business meaning

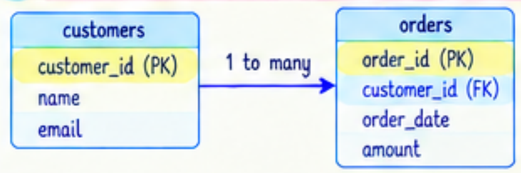
6. SCENARIO - WHY SURROGATE KEY ?

Scenario: A customer changes their email and city multiple times. If we use customer_id as primary key, we cannot store multiple historical records (SCD Type 2).

Solution:

- Use customer_sk as primary key.
- Keep customer_id as business key (with unique constraint).
- Each version gets a new surrogate key.
- Maintain effective_from, effective_to and is_current.

7. KEY CONSTRAINTS & DELETE BEHAVIOR (FOREIGN KEY)



Delete Behavior

- RESTRICT / NO ACTION – Prevents delete if child exists
- CASCADE – Deletes child records automatically
- SET NULL – Sets foreign key to NULL (if allowed)

8. COMPOSITE KEY vs PARTIAL KEY (EXAMPLES)

Composite Key (Student Enrollment)

student_id	course_id	grade
S101	C01	A
S101	C02	B
S102	C01	A

(student_id, course_id) together uniquely identify a record.

Partial Key (Weak Entity)

employee_id	dependent_no	name
E101	1	Ravi
E101	2	Priya
E102	1	Aman

dependent_no is unique only within an employee. Composite key: (employee_id, dependent_no).

9. QUICK REVISON NOTES

- ✓ Every candidate key is a super key, but not every super key is a candidate key.
- ✓ Primary key is one of the candidate keys.
- ✓ Alternate key is a candidate key not chosen as PK.
- ✓ Composite key = multiple columns.
- ✓ Partial key = unique only within parent entity.
- ✓ Foreign key → references PK/unique key of another table.
- ✓ Unique key → no duplicates (NULL handling varies by DBMS).
- ✓ Secondary key → used for search, not necessarily unique.
- ✓ Surrogate key → artificial, no business meaning, used for stability and SCD history.

2. PRACTICAL USE OF KEYS & SURROGATE KEYS (INTERVIEW READY)

Focus: ETL / SCD / dedup / idempotency / FK resolution / data quality / scenario questions

1. BUSINESS KEY vs SURROGATE KEY IN PIPELINES

Aspect	Business Key	Surrogate Key
Meaning	Identifies the real entity / event from the source system.	Identifies the warehouse row / version in the target system.
Source	Comes from source data (natural key).	Generated in data warehouse (ETL/ELT process).
Example	customer_id, order_id, (source_system, customer_id)	customer_id, order_sk (101, 102, ...)
Use in ETL	Used for deduplication, merge and matching.	Used as primary key in dimension and fact tables.
Stability	Can change (source dependent).	Stable and does not change.
Role in SCD Type 2	Identifies the same business entity across versions.	Unique for each version/row of the entity.

⚠ **Note:** A surrogate key does NOT prevent duplicate business records. Dedup must use business key + grain (e.g. source_system, customer_id).

2. DEDUPLICATION & IDEMPOTENT LOADS

Typical Pipeline Flow

Source file (batch) → Identify business key & grain → Deduplicate batch (remove dups) → MERGE / UPSERT into target → Rerun safe (no extra rows)

Example Business Keys (choose based on grain)

- (source_system, order_id, line_number) → for order line level
- (source_system, customer_id) → for customer level

Checklist for Deduplication & Idempotent Loads

- ✓ Define the grain first (what makes a record unique).
- ✓ Remove duplicate rows within the batch (using business key + grain).
- ✓ Use MERGE / UPSERT using business key (not surrogate key).
- ✓ Rerunning the same input file should not create extra rows.
- ✓ Validate row counts and important totals (e.g. amount) after rerun.

⚠ **Warning:** Using a new surrogate key for every row will still allow duplicate business records. Always deduplicate using the business key + grain.

3. FACT TO DIMENSION KEY RESOLUTION (SCD TYPE 2)

staging_orders (fact data)
- order_id
- customer_id
- order_ts
- amount

Join on business key + effective dates →

dim_customer (SCD Type 2)
- customer_sk
- customer_id
- city
- effective_from
- effective_to
- is_current

Example SQL (assign correct customer_sk based on event time)

```
SELECT o.order_id, d.customer_sk
FROM staging_orders o
LEFT JOIN dim_customer d
   ON o.customer_id = d.customer_id
  AND o.order_ts >= d.effective_from
  AND o.order_ts <  d.effective_to;
```

Key Points

- Facts store the surrogate key (e.g. customer_sk).
- Lookup uses business key + effective dates.
- Assign the version valid at the event time.
- If no match, use unknown member or inferred member process.

Example: Customer moved from Delhi → Bengaluru

customer_sk	customer_id	city	effective_from	effective_to	is_current
1001	C001	Delhi	2024-01-01	2024-06-30	N
1002	C001	Bengaluru	2024-07-01	9999-12-31	Y

Order on 2024-03-15 (for customer_id = C001) → should use customer_sk = 1001 (Delhi row)

4. SURROGATE KEY GENERATION METHODS & PITFALLS

Method	Good for	Risk / Note
Identity / Sequence	• Simple and widely supported in DB platforms. • Generates incremental values.	• Easy to implement and stable. • May require central sequence in distributed systems.
UUID (e.g. UUID4)	• Globally unique across systems. • Useful in distributed / parallel loads.	• Larger key size. • Not human-friendly. • Random UUID may not be sequential (can affect storage).
Hash (e.g. MD5/SHA)	• Useful for change detection or deterministic IDs. • Can generate key based on business columns.	• Very low but possible hash collisions (use strong hash). • Key changes if input changes. • Good for deterministic keys (e.g. hash of business key).
Row Number / Window Logic	• Can be used for key assignment within a batch (e.g. ROW_NUMBER in Spark/SQL).	• Risky across reruns or parallel loads (ordering can change). • May create unstable or overlapping keys. • Not recommended as the only method for production keys.

⚠ **Note:** Key generation must be deterministic or transactionally safe. Avoid methods that can create different keys for the same record on reruns.

5. DATA QUALITY & VALIDATION CHECKS

Check	What to Validate
✓ Primary / business key uniqueness	No duplicate business keys at the intended grain (e.g. (source_system, order_id).
✓ No duplicate business keys for current-only tables	Each business key should appear only once (e.g. no duplicates in active records).
✓ One current row per business key in SCD2	Exactly one row with is_current = Y for each business key.
✓ No overlapping effective date ranges	Effective date ranges for a business key should not overlap.
✓ Foreign keys in fact resolve to dimension.	All fact foreign keys should match a valid dimension key (no missing lookups).
✓ Unknown records tracked separately	Track and monitor unknown members (e.g. customer_sk = -1 / 0).
✓ Parent-child referential integrity	Child records must have valid parent records (e.g. order → customer, order_line → order).
✓ Counts before/after merge are reasonable	Validate row counts and key metrics (e.g. sum of amounts) after each load.

⚠ **Note:** In some lakehouse systems, primary/foreign key constraints may not be enforced. Pipeline validations and data quality checks are important to ensure key integrity and correct relationships.

6. TOP 5 INTERVIEW SCENARIO QUESTIONS (WITH EXPECTED ANSWERS)

Q1. Duplicate customer_id but unique surrogate IDs – is table valid?
A: Valid only if it is an SCD2 / history model. Otherwise investigate duplicates. Check the grain, ensure one current row per business key and no overlapping dates.
Follow-up Trap: Are there multiple current rows?

Q2. Two source systems both send customer_id = 101 – how do you avoid wrong joins?
A: Use (source_system, customer_id) as the source business key or maintain a mastered crosswalk / mapping table to resolve to a single customer.
Follow-up Trap: What if the same customer exists in both systems?

Q3. Source resends yesterday's order file – how do you avoid duplicates?
A: Use business key dedup + idempotent MERGE/UPSERT. Surrogate key alone is not enough. Rerunning the same input should not create extra rows.
Follow-up Trap: How do you validate idempotency?

Q4. Late-arriving March order comes after customer changed city in June – which customer_sk do you assign?
A: Assign the historical version valid at the March event time (using effective dates).
Follow-up Trap: What if no matching history is found?

Q5. You generate surrogate keys with ROW_NUMBER in Spark and run jobs in parallel – what can go wrong?
A: Keys can be unstable or overlapping because ordering can change across runs or parallel jobs. Prefer sequence/identity/UUID or controlled key assignment.
Follow-up Trap: Will the keys be the same on rerun? Why/why not?

7. QUICK INTERVIEW TAKEAWAYS

- ✓ Primary key / unique key enforce row-level uniqueness (when supported).
- ✓ Foreign key enforces relationship or must be validated in pipeline.
- ✓ Business key is used for deduplication and merge (identifies the real entity/event).
- ✓ Surrogate key is used for warehouse identity and SCD history (unique per row/version).
- ✓ Define the grain first, then choose the right keys.
- ✓ Always think about reruns (idempotency), late-arriving data, and data quality validation.